

Posibles issues de Arranque

1. ⚠ Error de Configuración de Testing (Contexto de Spring)

El primer error que encuentres es un problema de configuración cuando intentabas ejecutar un test de integración:

java.lang.IllegalStateException: Unable to find a @SpringBootConfiguration by searching packages upwards from the test.

🔴 El Problema: Paquetes Desalineados

Este error no es de Postman, sino del entorno de **Spring Test**. Ocurre porque Spring no logra encontrar automáticamente tu clase principal con la anotación **@SpringBootApplication** (que también es una **@SpringBootConfiguration**) para cargar el contexto de aplicación mínimo que requiere el test.

Generalmente, esto sucede por una de estas razones:

1. **Estructura de Paquetes Incorrecta:** La clase de test (TaskControllerTest.java) se encuentra en un paquete que no es un subpaquete del paquete de la clase principal (TasksApplication.java).
2. **Confusión de Ejecución:** El IDE está intentando ejecutar el archivo de test como si fuera la aplicación completa.

🔧 *Solución Sugerida*

Para la mayoría de los proyectos Spring Boot, la solución es la **estructura de paquetes**:

- **Clase Principal:** com.ejemplo.tasks.TasksApplication
- **Clase de Test:** com.ejemplo.tasks.controller.TaskControllerTest

Asegurarse de que el test esté en un subpaquete del paquete base (com.ejemplo.tasks) permite a Spring detectar la configuración automáticamente.

2. 🚨 Error de API al Llamar desde Postman (HTTP 400)

El segundo error estaba relacionado con la forma en que el cliente (Postman) interactuaba con tu API:

HttpMessageNotReadableException: Required request body is missing...

🔴 El Problema: Petición HTTP Inválida

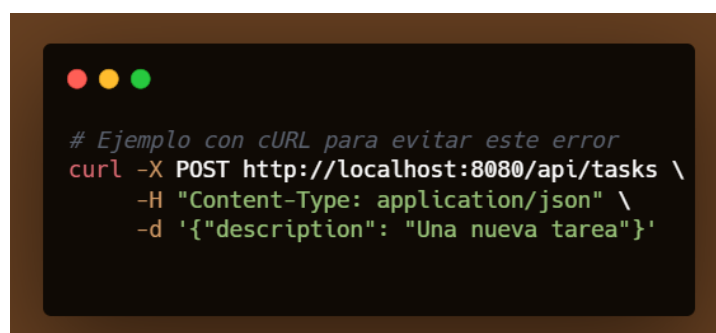
Este error indica que tu TaskController esperaba recibir un objeto JSON en el cuerpo de la petición (gracias a la anotación `@RequestBody`), pero la petición llegó sin cuerpo o con una cabecera incorrecta.

Causa: El cliente (Postman/curl) no envió la cabecera `Content-Type: application/json` o no incluyó los datos JSON en el *body*.

🔧 Solución Aplicada

La corrección se realiza en el cliente: configurar la herramienta para que envíe el JSON con la cabecera adecuada:

Bash



```
# Ejemplo con cURL para evitar este error
curl -X POST http://localhost:8080/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"description": "Una nueva tarea"}'
```

3. 📉 Problemas de Respuesta de la API (200 vs 201)

Una vez que la petición HTTP fue válida, la aplicación creó la tarea, pero reportó dos problemas con la respuesta: el código HTTP fue 200 OK y el campo `completed` era `false`.

A. Problema de Estado HTTP: No se devuelve 201 Created

El protocolo HTTP recomienda el código **201 Created** para las peticiones POST exitosas que resultan en la creación de un nuevo recurso. Si recibías **200 OK**, significa que Spring usó el estado por defecto.

Solución al Problema del Código HTTP (No 201)

Para garantizar el código 201 Created, debes ser explícito en tu TaskController.java. Hay dos métodos principales:

Opción 1 (Recomendada): Usar `ResponseEntity` y `HttpStatus.CREATED`

Este es el enfoque más profesional y flexible:

Java

```
@PostMapping
public ResponseEntity<Task> createTask(@RequestBody Task task) {
    Task createdTask = taskService.createTask(task);
    // SOLUCIÓN: Usar HttpStatus.CREATED para el código 201
    return new ResponseEntity<>(createdTask, HttpStatus.CREATED);
}
```

Opción 2: Usar solo `@ResponseStatus` (Menos flexible)

Java

```
// ...
@PostMapping
@ResponseStatus(HttpStatus.CREATED) // <-- Esto fuerza el 201
public Task createTask(@RequestBody Task task) {
    return taskService.createTask(task);
}
```

B. Problema de Datos: `completed: false`

Respecto al `completed: false`, este es el **comportamiento esperado y correcto** del modelo de datos de Java:

- En tu clase `Task.java`, definiste: `private boolean completed = false;`.
- Cuando Spring deserializa el JSON y el cliente **no incluye** el campo `"completed"`, Java lo inicializa a su valor por defecto de instancia: `false`.

Conclusión: La tarea se guarda correctamente como incompleta. Para que una nueva tarea se cree como `true` (Completada), el cliente debe enviarlo explícitamente en el cuerpo de la petición POST:

JSON



```
{
  "description": "Tarea ya resuelta",
  "completed": true
}
```